

【专栏二：深度学习02】 - 【一张图讲清楚：什么是向前传输和向后传输】



快乐飞猪ing



· 码龄7年 · 粉丝可见 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-27 12:01:22 · 594 阅读



AI 写同款 · GEO 优化

深度学习 - 23 Whisper LoRA

Dropout(X)这一部分尤其需要和普通数据增强区分开。Base:LoRA:进入 LoRA 分支的 hidden-state activ...

[阅读详情 >](#)

前言

上一篇文章里，我用“让机器学会认出猫”的案例，讲了什么是深度学习。

但写完之后，一个更关键的问题其实才刚刚开始：**模型到底是怎么学会这些规律的？**

它不是人，不会“看着看着就懂了”。

那它到底是怎么从一开始什么都不会，到最后慢慢变得越来越准的？要理解这个过程，就绕不开四个关键词：

- 向前传播
- 损失函数
- 向后传播
- 参数更新

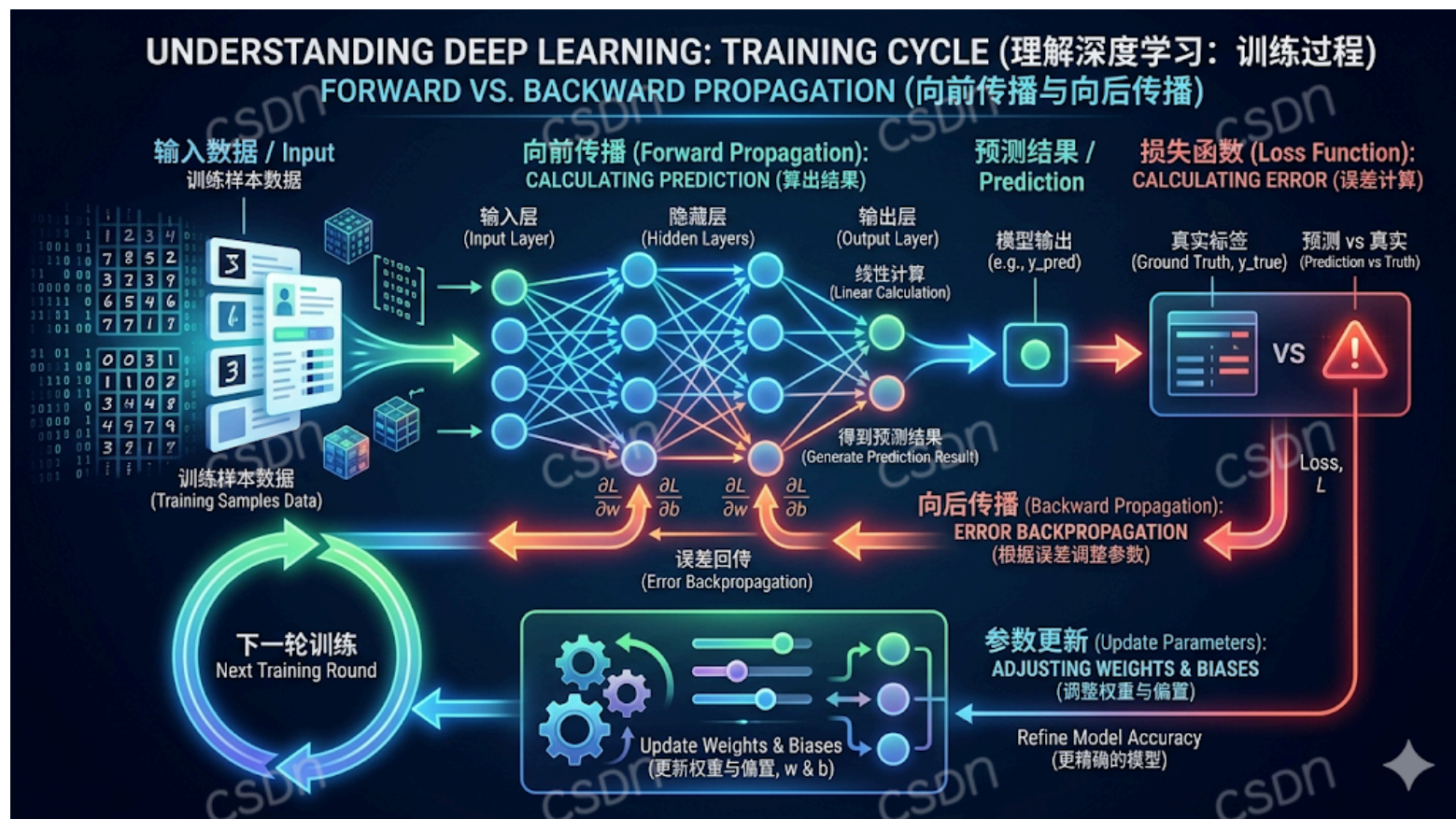
内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

这篇文章，我就结合下面这张图，把深度学习训练过程完整梳理一遍。



如果只用一句话总结这张图，我会这样说：

向前传播负责“先算出答案”，损失函数负责“看错多少”，向后传播负责“把错误往回传”，参数更新负责“让模型下次更接近正确答案”。

整套训练流程其实就是一个不断循环的闭环：

输入数据 → 向前传播 → 预测结果 → 损失函数 → 向后传播 → 参数更新 → 下一轮训练

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

你可以先把这个主线记住。

后面无论公式还是代码，本质上都在围绕这个闭环展开。

一、输入数据：训练从样本开始

深度学习训练的第一步，不是公式，也不是代码，而是：

先有数据。

模型不可能凭空学习。它必须先看到样本，才能从中总结规律。

比如在“认猫”的场景里，输入数据可以是：

- | | |
|---|-----------|
| 1 | 很多猫的图片 |
| 2 | 很多非猫的图片 |
| 3 | 每张图片对应的标签 |

而在更一般的任务里，输入数据也可以是：

- | | |
|---|------|
| 1 | 文本 |
| 2 | 语音 |
| 3 | 表格数据 |
| 4 | 数值特征 |

图里的“输入数据”其实是在强调一件非常基础但很重要的事：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

模型学的不是概念本身，而是数据里的模式。

也就是说，深度学习从来不是“先定义规则再执行”，而是：

先给数据

再让模型从数据里学

二、向前传播：模型先算出一个预测结果

接下来进入第一步真正的计算过程：

向前传播 (Forward Propagation)

这个名字听起来很专业，但如果换成更直白的话，其实就是：

让输入数据从前往后走一遍网络，最后算出一个预测结果。

比如你把一组输入送进模型后，它会一层一层往前计算，最后输出一个结果：

- 1 是猫的概率
- 2 属于哪个类别
- 3

内容来源: csdn.net AI写代码 |

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页: https://blog.csdn.net/m0_46279060

所以**向前传播的核心目标**非常简单：

模型先猜一个答案出来。

三、先把第一个公式讲明白：为什么会有 $z = Wx + b$ ？

很多人一看到深度学习公式就紧张，其实第一条公式没有那么可怕。最常见的形式就是：

$$z = Wx + b$$

我们先不要急着看符号，先看它在表达什么。

它其实只是在说：

输入*权重，即模型会把输入乘上不同权重，再加上一个偏置，得到当前层的计算结果。

你可以把它理解成：

- | | |
|---|--------------------|
| 1 | x: 输入数据 |
| 2 | W: 每个输入的重要程度，也就是权重 |
| 3 | |
| 4 | |

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页: https://blog.csdn.net/m0_46279060

b: 一个整体调整项，也就是偏置
z: 这一层算出来的中间结果

—————下面是一个非常具体的小例子—————

内容来源: [csdn.net](https://blog.csdn.net)

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页: https://blog.csdn.net/m0_46279060

从特征提取到模型计算：极简抽象示例



特征提取与
赋数值

假设我们现在不直接处理整张猫图，而是先做一个极简抽象：



耳朵特征强度

$$x_1 = 0.8$$



胡须特征强度

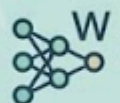
$$x_2 = 0.6$$

输入向量

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}$$

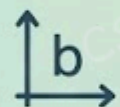
模型参数设定

假设模型当前的权重和偏置：



$$W = \begin{bmatrix} 0.5 & 0.3 \end{bmatrix}$$

$w_1=0.5, w_2=0.3$



$$b = 0.1$$

数值代入与计算



计算过程

那这时候：

$$z = Wx + b$$

代入去程：

$$\begin{bmatrix} 0.5 & 0.3 \end{bmatrix} \cdot \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix} + 0.1$$
$$(0.5 \times 0.8) + (0.3 \times 0.6) + 0.1$$

内容来源：csdn.net

作者昵称：快乐飞猪

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

计算结果与结论

$$0.4 + 0.18 + 0.1$$

0.68

$$z = 0.68$$

模型计算结果：

这个 0.68 就是模型当前算出来的一个中间结果。

这一步的直觉是什么？

这一步本质上是在做：

根据输入特征，按不同重要程度加权打分。

比如这里你可以粗略理解成：

- 1 耳朵特征贡献了 0.4
- 2 胡须特征贡献了 0.18
- 3 偏置再整体补了 0.1

最后模型当前对“这是猫”这件事，先打出了一个 0.68 的中间分数。

所以 $z = Wx + b$ 并不神秘，它只是：**把输入特征做一次加权求和。**

AI写代码 |

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

四、只有线性计算还不够，所以还需要激活函数

内容来源: [csdn.net](https://blog.csdn.net/m0_46279060)

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159511910

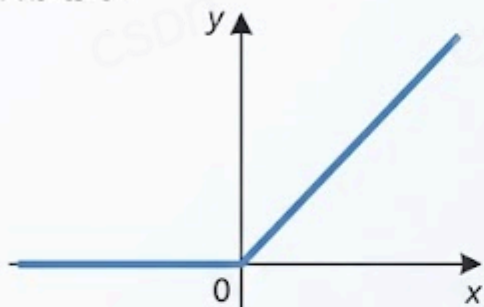
作者主页: https://blog.csdn.net/m0_46279060

最常见的两个激活函数

1. ReLU (整流线性单元)

$$\text{ReLU}(x) = \max(0, x)$$

函数图像:



 输入 < 0
→ 输出 = 0

 输入 ≥ 0
→ 输出 = 输入

这个公式看起来复杂一点,但你不用先记。

它会把一个数压缩到 0 到 1 之间。

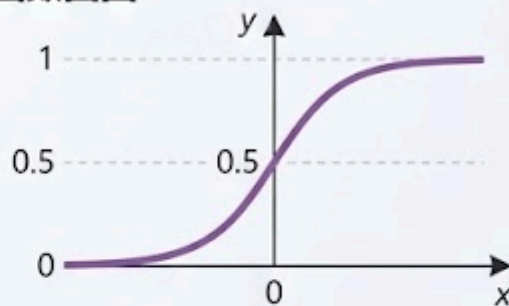
例如: $\text{ReLU}(-2) = 0$ | $\text{ReLU}(3) = 3$

- 很大的正数 → 接近 1
- 很大的负数 → 接近 0
- 中间值 → 落在 [0, 1] 之间

2. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

函数图像:



核心作用: 将数值压缩至 [0, 1] 范围之内。

核心作用: 将数值压缩至 [0, 1] 范围之内。

比如:

- 很大的正数 → ≈ 1
- 很大的负数 → ≈ 0
- 中间值 → 落在 [0, 1] 之间

所以它特别适合做“概率输出”。

比如刚刚我们算出来：



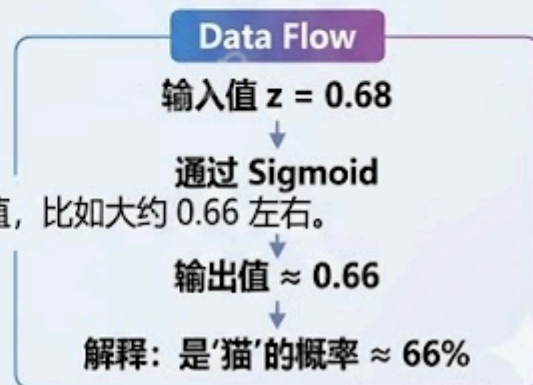
$$z = 0.68$$

如果通过 sigmoid，结果会变成一个 0 到 1 之间的值，比如大约 0.66 左右。

这时你就可以把它解解成：

模型认为“这是猫”的概率大约是 66%。

使用场景：
特别适合用于‘概率输出’。



如果网络每一层都只是简单做 $z = Wx + b$ ，那问题来了：

无论堆多少层，本质上都还是线性计算。

这样模型就学不会复杂模式，所以神经网络里还要再加一个关键角色：

激活函数 (Activation Function)

它的作用可以先简单理解成：**给模型加入非线性能力**，模型不只是机械加权，而是能够**表达更复杂的关系**。

——最常见的两个激活函数——

1. ReLU

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

公式是：

$$ReLU(x) = \max(0, x)$$

这是什么意思？很简单：

...

如果输入是负数，就变成 0

如果输入是正数，就原样保留

例如：

$$ReLU(-2) = 0$$

$$ReLU(3) = 3$$

它的好处是**简单、有效**，所以在**隐藏层**里非常常见。

2. Sigmoid

公式是：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

这个公式看起来复杂一点，但你不用先记形式，先记作用：它会**把一个数压缩到 0 到 1 之间**。

比如：

很大的正数 → 接近 1

很大的负数 → 接近 0

中间值 → 落在 0 到 1 之间

所以它特别适合做“**概率输出**”。

比如刚刚我们算出来：

$$z = 0.68$$

如果通过 sigmoid，结果会变成一个 0 到 1 之间的值，比如大约 0.66 左右。

这时你就可以把它解释成：

模型认为“这是猫”的概率大约是 66%。

五、预测结果：模型先给出自己的答案

经过向前传播以后，模型就会先给出一个**预测值**。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

比如：

真实标签 $y = 1$ ，表示“这就是猫”

模型预测值 $\hat{y} = 0.66$

这里的 $\hat{y}$ 读作 “y hat”，表示预测值。

这时候模型已经完成了一件事：

它先给出了自己的判断。

注意，这个判断可能是对的，也可能是错的。

但无论如何，它必须**先给出一个答案，后面才能知道自己错了多少。**

六、损失函数：模型怎么知道自己错了多少？

有了预测值之后，下一步就是：

把预测值和真实值进行比较。

这时候就要用到 **损失函数（Loss Function）**。

损失函数的作用非常好理解：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

它用来衡量模型这次预测错了多少。

你可以把它想成一次考试后的“扣分机制”：

- 如果模型预测得很接近真实答案，损失就小
- 如果模型偏得很远，损失就大

——最简单的理解方式——

如果真实值是：

$$y = 1$$

模型预测是：

$$\hat{y} = 0.66$$

那模型虽然方向没错，但还不够准，它离真正的 1 还有差距。

这个差距，就会通过损失函数被量化出来。

你现在不需要先把所有损失函数公式背下来，只要先抓住一句话：

损失函数负责把“模型错了多少”变成一个可以计算的数。

这一步特别关键，因为它是向前传播和向后传播之间的桥梁。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

七、向后传播：把误差信息从后往前传

现在模型已经知道自己错了,但仅仅知道“错了”,还不够。

更关键的问题是：

到底是哪些参数导致它错了？

这些参数应该往哪个方向改？

这时就轮到 **向后传播 (Backward Propagation)** 出场了。

你可以先把向后传播理解成：

把“错了多少”这个信息，从输出层一层层往回传。

它的目标不是重新算一遍答案，而是找出：

哪些参数影响了最终结果

每个参数应该改多少

八、先把“梯度”这个概念具体化

很多人一听反向传播就会被“梯度”吓到。

其实你可以先把梯度理解成一句很朴素的话：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

如果我把某个参数改一点点，损失会变大还是变小？

如果某个参数稍微增大，会让损失明显变大，说明这个方向不对。

如果某个参数稍微调整，会让损失变小，说明这个参数可以往这个方向改。

这就是梯度想表达的核心含义。

图里你已经放了这两个符号：

$$\frac{\partial L}{\partial w} \quad \frac{\partial L}{\partial b}$$

它们可以先这样理解：

- $\frac{\partial L}{\partial w}$ ：损失对权重 w 的变化有多敏感
- $\frac{\partial L}{\partial b}$ ：损失对偏置 b 的变化有多敏感

所以反向传播，其实就是在回答：

每个参数对错误负责多少？

九、为什么误差要“从后往前传”？

因为最终的损失，是在输出层才算出来的。

也就是说：

- 先有输出

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

- 再有预测误差
- 所以误差信息必须从最后一层开始往回传

这也是“向后传播”这个名字的来源，它不是随便倒着算，而是在做一件非常具体的事：

沿着网络的反方向，把误差一层层传回去，再逐层计算各参数的梯度。

如果你后面要继续深入数学，这里就会自然接到 **链式法则**。

不过在这篇科普里，你先记住直觉就够了：

反向传播 = 从结果出发，反过来找每一层该怎么改。

十、参数更新：模型真正“学会”的地方

向后传播算出梯度之后，下一步就是**参数更新**。

这一部分才真正对应“**学习**”这件事：

因为模型所谓的“会了”，本质上不是它突然理解了世界，而是它内部的参数，被一点一点调得更适合当前任务。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

最经典的参数更新公式是：

$$W := W - \eta \frac{\partial L}{\partial W}$$
$$b := b - \eta \frac{\partial L}{\partial b}$$

这两个公式看起来有点抽象，我们先把它翻译成人话：

- W 和 b ：是当前参数
- $\partial L / \partial W$ 和 $\partial L / \partial b$ ：是梯度，也就是参数该怎么改的方向信息
- η ：学习率，决定每次改多大

这一步的直觉非常重要：

参数更新，就是沿着“让损失变小”的方向去调参数。

你可以把学习率 η 理解成“步子大小”：

- 步子太大，可能会跨过最优点
- 步子太小，训练会特别慢

所以训练过程并不是一次就成功，而是很多轮“小步调整”的累积。

十一、下一轮训练：为什么训练是一个循环

参数更新完之后，训练不会立刻结束。

模型会带着“更新后的参数”，重新进入下一轮：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

- 1 再跑一次向前传播
- 2 再算一次预测
- 3 再计算一次损失
- 4 再做一次向后传播
- 5 再更新一次参数

于是，这就构成了一个完整闭环：

输入数据 → 向前传播 → 预测结果 → 损失函数 → 向后传播 → 参数更新 → 下一轮训练

这也是为什么深度学习训练不是一次性过程，而是一个不断重复的 Q 循环。

你可以把它理解成：

模型并不是一下子学会，而是在“先预测—再纠错—再调整”的循环里，慢慢逼近更好的结果。

十二、用一句话串起整张图

现在再回头看这张图，它其实想表达的就是这件事：

- 输入数据进入模型
- 向前传播先算出预测结果
- 损失函数计算模型错了多少
- 向后传播把误差信息往回传

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

- 参数更新让模型下一次更接近正确答案
- 整个过程不断循环，模型就慢慢学会了规律

所以如果让我把整张图压缩成一句话，我会这样总结：

向前传播负责算结果，损失函数负责量化错误，向后传播负责找出哪里该改，参数更新负责让模型下一次更准。

写在最后

到这里，其实你已经抓住了深度学习训练最核心的闭环。

很多人一开始学深度学习，会被大量术语吓到：

- 前向传播
- 反向传播
- 梯度
- 损失函数
- 参数更新

但如果把它们放回训练流程里看，本质并不神秘：

先猜一个答案，再看错了多少，然后根据错误去调整自己。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页：https://blog.csdn.net/m0_46279060

这就是深度学习训练过程最核心的逻辑。

提问

为什么神经网络里一定要有激活函数？

如果只有线性计算，多层网络为什么学不出复杂模式？

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159511910

作者主页: https://blog.csdn.net/m0_46279060